

Vorlesung Rechnerarchitekturen (AIN 2 – WS 2021/2022) Klausur (K90)

Datum: 08. Februar 2022, 14:00 Uhr

Bearbeitungszeit: 90 Minuten (Online)

Gesamtpunkte: 75

Mindestpunktzahl zum Bestehen der Klausur: 30

Name:

Matrikelnummer:

Semester:

Aufgabe	1	2	3	4	5	6	7	Gesamt
Punkte	7	5	8	12	16	12	15	75

Hinweise:

- Alle Aufgaben müssen handschriftlich bearbeitet werden.
- Bei jeder angegebenen Lösung muss der Lösungsweg nachvollziehbar sein.
- Nach Ende der Klausur müssen Sie eine Selbstständigkeitserklärung (auf Moodle) abgeben, dass Sie die Klausur eigenständig und nur mit den erlaubten Hilfsmitteln erstellt haben.
- Erlaubte Hilfsmittel sind alle außer Quellen im Internet, die eine explizite Lösung oder Teillösung zu Klausuraufgaben bereitstellen, insbesondere natürlich von Dritten bereitgestellt Lösungen oder über das Internet verfügbare Programme, die Aufgaben oder Teilaufgaben lösen. Allgemeine Dienste wie z.B. Internet-Taschenrechner, Python-IDE, Binär-Dezimalrechner, etc. sind erlaubt.
- Abgabe in einem PDF mit Dateinamen „RA_<Matrikelnummer>_<Nachname>.pdf“

Aufgabe 1: Assembler Programmierung (7 Punkte)

Implementieren Sie die folgende Funktion in MIPS Assembler. Halten Sie sich bei der Implementierung des Assembler-Code strikt an die Vorgaben der C-Funktion sowie die MIPS Konventionen. (10P)

```
unsigned char g(unsigned char b, unsigned char a) {  
    unsigned char A[20];  
    unsigned char j;  
    A[19]=b+20;  
    for(j=18; j>0;j--) {  
        A[j]=A[j+1]+a;  
    }  
    return A[0];  
}
```

Aufgabe 2: Assembler Instruktionen (5 Punkte)

Die folgenden Tabellen enthalten eine Reihe von Instruktionen, die Sie nacheinander für die ebenfalls in den Tabellen gegebenen Register- und Speicherinhalte ausführen sollen. Tragen Sie die Veränderungen der gelisteten Register- und Speicherinhalte jeweils in den freien Feldern der Tabellen ein.

Hinweise:

- Punkte pro Instruktion wie in der ersten Spalte der Tabelle angegeben.

		Register (Inhalte als Signed Integer)						
		\$s0	\$s1	\$s2	\$t0	\$t1	\$t2	\$sp
P	Instruktionen	-16	4	2	13	21	-2	0x7FFF AF08
0,5	sra \$t1, \$s0, \$s1					-1		
1,0	sw \$t2, -5(\$t0)							
0,5	slt \$t2, \$s0, \$s2						1	
1,0	lw \$s1, 2(\$s2)		273					
0,5	sllv \$t0, \$s0, \$s2				64			
1,0	sb \$s2, 4(\$sp)							
0,5	addi \$s0, \$s0, 2	-14						

Speicherausschnitt					
Adresse (hexadezimal)	Inhalt (unsigned Bytes)		Adresse (hexadezimal)	Inhalt (unsigned Bytes)	
...		Änderung	...	...	Änderung
0x0000 000B	0	255	0x7FFF AF13	255	
0x0000 000A	2	255	0x7FFF AF12	255	
0x0000 0009	4	255	0x7FFF AF11	253	
0x0000 0008	16	253	0x7FFF AF10	3	
0x0000 0007	0		0x7FFF AF0F	2	
0x0000 0006	0		0x7FFF AF0E	1	
0x0000 0005	1		0x7FFF AF0D	0	
0x0000 0004	17		0x7FFF AF0C	42	2
0x0000 0003	255		0x7FFF AF0B	7	
0x0000 0002	255		0x7FFF AF0A	128	
0x0000 0001	8		0x7FFF AF09	124	
0x0000 0000	0		0x7FFF AF08	0	

Name:

Matrikelnummer:

Aufgabe 3: Maschinensprache (8 Punkte)

Im Folgenden ist ein Stück Programm-Code sowohl in Assemblersprache als auch in Maschinensprache gegeben. Beide Programm-Codes weisen Lücken auf. Ergänzen Sie diese Lücken.

Hinweis: Punkte pro Instruktion in der rechten Spalte.

Verwenden Sie für diese Aufgabe die folgenden **OPCODES** und **FUNCTION** Codes:

Code (binär)	000001	000010	000011	000100	000101
Code ist OPCODE von Instruktion	addi	slti	ori	andi	nori
Code ist FUNCT von Instruktion	add	sub	slt	sllv	or
Code (binär)	000110	000111	001000	001001	001010
Code ist OPCODE von Instruktion	lw	sw	beq	bne	j
Code ist FUNCT von Instruktion	and	nor	sra	sll	srl

Verwenden Sie die folgende Registeradressierung:

Nummer	Adresse	Register	Nummer	Adresse	Register	Nummer	Adresse	Register
4	00100	\$t0	8	01000	\$s0	12	01100	\$a0
5	00101	\$t1	9	01001	\$s1	13	01101	\$a1
6	00110	\$t2	10	01010	\$s2	14	01110	\$a2
7	00111	\$t3	11	01011	\$s3	15	01111	\$a3

Füllen Sie die folgenden Lücken in Assembler- bzw. Maschinencode aus:

Speicher- adresse	Maschinenformat								Assembler	P
16	0000	01__	0110	1001	1111	1111	1111	1111	L1: addi \$s1 \$s3 ⁻¹	2
20	0001	10__	1000	0101	0000	0000	0000	0001	L2: lw \$t1 1(\$t0)	1
24	0010	0001	0100	0101	0000	0000	0000	0001	L3: beq \$s2 \$t1 ^{L4}	1
28	0000	0000	1000	0100	0110	0010	1000	1001	L4: sll \$a0 \$t0 ¹⁰	1
32	0000	0000	1100	1000	0100	1000	0000	0100	L5: sllv \$a0 \$s0 \$t2	2
36	0010	10__	0000	0000	0000	0000	0000	0100	L6: j L1	1

Aufgabe 4: Datenpfad und Steuerwerk (12 Punkte)

4.1 In Abbildung 4 sind Datenpfad und Steuerwerk einer Eintakt-Implementierung der MIPS CPU dargestellt. Der Inhalt der Register ist in Tabelle 4.1 gegeben. (6P)

Einige Daten- und Steuersignale sind mit Nummern bzw. Buchstaben in Rechtecken versehen. Tragen Sie in Tabelle 4.1 die Werte ein, die diese Signale bei der Ausführung der Instruktion

Maschinencode:

Binär: 0001 0000 1010 1010 0101 1001 0111 0100

Hex: 10 AA 59 74

R-Format (binär): 000100 00101 01010 01011 00101 110100

R-Format (dezimal): 4 5 10 11 5 52

I-Format (binär): 000100 00101 01010 0101100101110100

I-Format (dezimal): 4 5 10 22900

J-Format (binär): 000100 00101010100101100101110100

J-Format (dezimal): 4 11164020

im nächsten Takt annehmen. Die Instruktion steht im Speicher an Adresse 67000.

Hinweise:

- Sie können die Signale in Binär-, Dezimal- oder Hexadezimaldarstellung angeben.
- Alle **nicht relevanten** Steuerbefehle werden auf 0 gesetzt.
- Der Ausgang des Datenspeichers ist nur bei Ladebefehlen ungleich 0.
- In Tabelle 4.2 und Tabelle 4.3 sind die relevanten ALU Steuerbefehle gegeben.
- Der Code steht noch einmal auf der nächsten Seite

4.2 Eine MIPS CPU soll genau die folgenden Befehle unterstützen:

`andi, ori, nor, and, xori, beq, sub, bne, nori`

- a. Wie viele Bits muss dann das ALUOp Steuersignal (Control → ALU Control) mindestens enthalten? Erklären Sie ihre Antwort. (1,5P)

4

- b. Wie viele Bits muss der ALU Steuerbefehl (ALU Control → ALU) dazu mindestens enthalten? Erklären Sie ihre Antwort. (1,5P)

2

4.3 Erweitern Sie Abbildung 4 so, dass die Instruktion `sra` unterstützt wird. (3P)

Name:

Matrikelnummer:

0	9	4	9	8	8	12	8	16	6	20	10	24	7	28	9
1	7	5	10	9	10	13	10	17	6	21	10	25	7	29	9
2	7	6	7	10	7	14	7	18	6	22	7	26	6	30	10
3	6	7	8	11	8	15	9	19	7	23	6	27	6	31	8

Tabelle 4.1: Registerinhalte in Hexadezimaldarstellung

ALU operation	Function	ALU operation	Function
0000	AND	0110	Subtract
0001	OR	0111	set-on-less-than
0010	Add	1100	NOR

Tabelle 4.2: Übersicht ALU Control

Instruction	ALUOp
lw, sw	00
Beq	01
R-Type	10

Tabelle 4.3: Übersicht ALUOp-Steuersignale

Daten- und Steuersignale					
1		4		A	
2		5		B	
3		6		regWrite	

Tabelle 4.4: Lösung

Maschinencode:

Binär: 0001 0000 1010 1010 0101 1001 0111 0100

Hex: 10 AA 59 74

R-Format (binär): 000100 00101 01010 01011 00101 110100

R-Format (dezimal): 4 5 10 11 5 52

I-Format (binär): 000100 00101 01010 0101100101110100

I-Format (dezimal): 4 5 10 22900

J-Format (binär): 000100 00101010100101100101110100

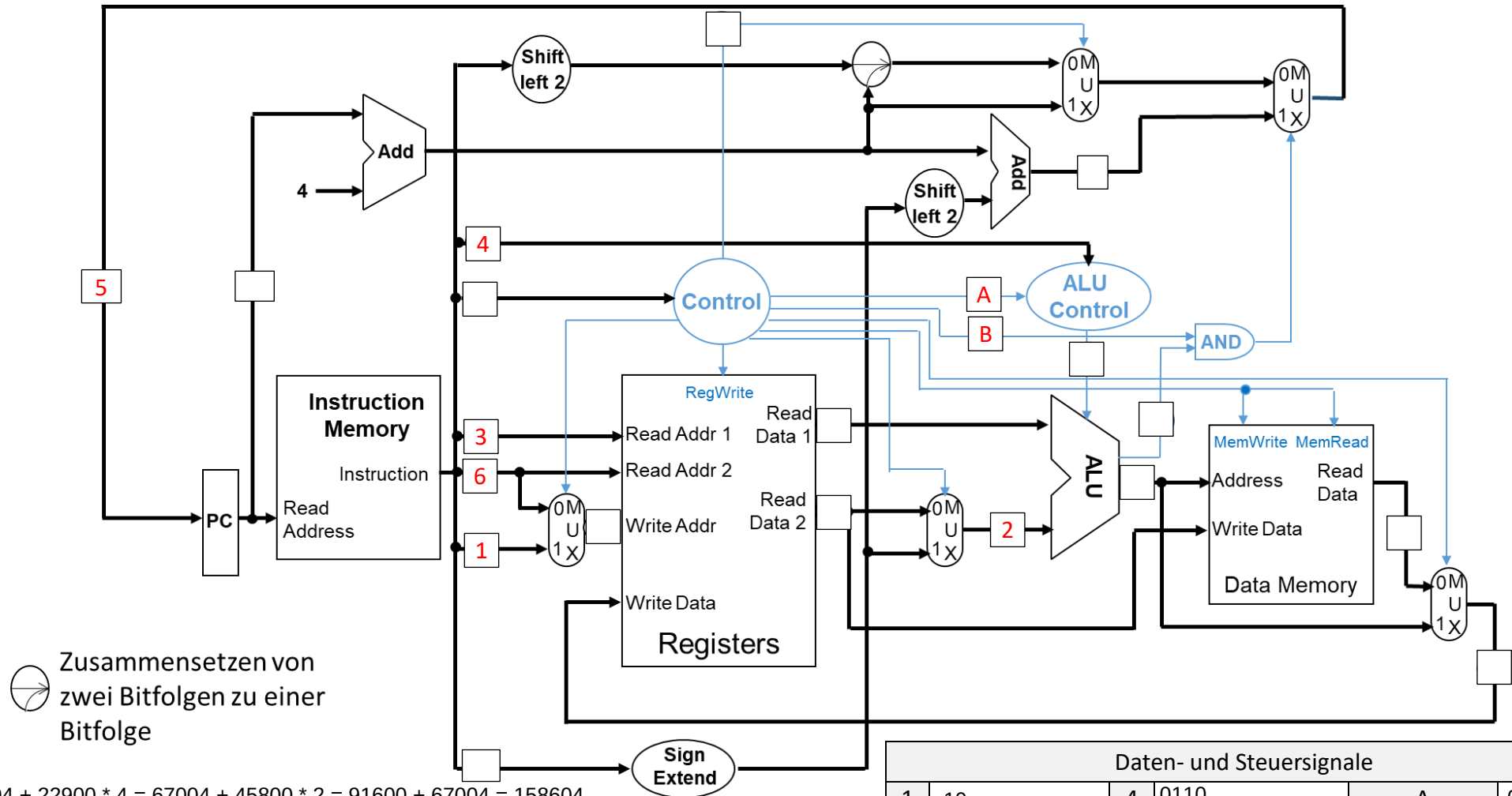
J-Format (dezimal): 4 11164020

Name:

Matrikelnummer:

R-Format (dezimal): 4 5 10 11 5 52

I-Format (dezimal): 4 5 10 22900



$$67004 + 22900 * 4 = 67004 + 45800 * 2 = 91600 + 67004 = 158604$$

Abbildung 4: Single-Cycle MIPS CPU

Daten- und Steuersignale					
1	10	4	0110	A	01
2	7	5	158604	B	0
3	5	6	10	regWrite	0

Aufgabe 5: Hazards (16 Punkte)

In dieser Aufgabe betrachten wir eine Multi-Cycle CPU mit Forwarding, auf der der Code in Tabelle 5 ausgeführt wird. Die CPU besitzt eine Einheit zur Branch-Prediction, so dass der neue Program-Counter bei Branch- und Jump-Instruktionen in der **IF-Stage** bestimmt und geschrieben wird. Außerdem verfügt die CPU über zusätzliche Hardware um Branch Bedingungen in der ID Stage auszuwerten.

5.1 Betrachten Sie die Data Hazards, die bei der Ausführung des Codes auftreten.

- a) Bestimmen Sie alle Data Hazards, die durch Forwarding beseitigt werden. Notieren Sie dazu in der mit „Forwarding“ bezeichneten Spalte das Forwarding in der Form $FW(\$x, R, S)$, wobei R das Pipeline-Register bezeichnet, aus dem der Operand genommen wird und S die Pipeline-Stage ist, in der das Register verwendet wird.

(7P)

Beispiel: $FW(\$s0, EX/MEM, EX)$

- b) Tragen Sie in die Spalte „Data Bubbles“ ein, um wie viele Takte sich die Ausführung des Programms bei Verwendung von Forwarding verzögert. (1,5P)
- c) Versuchen Sie, die verbleibenden Data Bubbles durch Code Scheduling zu vermeiden. (1,5P)

5.2 Betrachten Sie nun die Control Hazards, die bei der Ausführung des Codes auftreten. Gehen Sie jetzt davon aus, dass der neue Program-Counter bei Branch- und Jump-Instruktionen in der **ID-Stage** bestimmt und geschrieben wird.

- a) Tragen Sie in die Spalte „Control Bubbles“ die Anzahl der Takte ein, die aufgrund von Control Hazards verloren gehen,
- i) wenn bei dem Branch-Befehl in Zeile 3 gesprungen wird und
- ii) wenn bei dem Branch-Befehl in Zeile 3 nicht gesprungen wird. (2P)

Verwenden Sie folgendes Format: i) | ii)

- b) Beschreiben sie wo sich Branch Target Buffer und Branch Predictor in der CPU befinden und was ihre Aufgaben sind. (4P)

Name:

Matrikelnummer:

Tabelle 5: Code und Lösungs-Template zu Aufgabe 5

			Forwarding	Data Bubbles	Control Bubbles
1		add \$s2,\$s1,\$t0			
2		slti \$s1,\$t0,32			
3		bne \$s2,\$t0, L1	FW(\$s2,MEM/WB,ID)	1	
4		j L2			0 1
5	L1:	lw \$s3,32(\$s1)			1 0
6		add \$t0,\$s3,\$s1	FW(\$s3,EX/MEM,EX)		
7		lw \$t1,36(\$s2)			
8		sw \$t1,24(\$t0)	FW(\$t1,EX/MEM,EX), FW(\$t0,MEM/WB,MEM)		
9	L2:	add \$s2,\$t1,\$s1	FW(\$t1,MEM/WB,EX)		0 1
10		sw \$s2,16(\$s1)	FW(\$s2,EX/MEM,EX)		
			Summe:	1	1 2

Aufgabe 6: Pipeline (9 Punkte)

In Abbildung 6.2 ist die MIPS Pipeline mit Fokus auf die EX-Stage dargestellt. Der Code in Abbildung 6.1 wird auf der CPU ausgeführt. Die Register- und Speicherinhalte vor Ausführung von Zeile 1 sind in Tabelle 6.1 gegeben.

6.1 In der Darstellung der Pipeline finden Sie Zahlen in grau-markierten Kreisen. Tragen Sie in Tabelle 6.2 die entsprechenden Werte ein, wenn die **lw-Instruktion aus Zeile 5** in der EX-Stage ausgeführt wird. Dabei stehen die Zahlen für Pipeline-Register-Inhalte von ID/EX **vor** der Ausführung, Daten- und Steuersignale **während** der Ausführung und Pipeline-Register-Inhalte von EX/MEM **nach** der Ausführung der Instruktion in der EX-Stage. Gehen Sie davon aus, dass in dieser Aufgabe **keine Verzögerungen durch Control Hazards** auftreten. (9P)

Hinweise:

- In Abbildung 6.2 finden Sie auch Kopien der Abbildung 6.1 sowie der Tabellen 6.1 und 6.2, um eine einfachere Bearbeitung mit einem Bildschirm zu ermöglichen.
- Die Eingänge der Multiplexer sind von oben nach unten aufsteigend nummeriert. Geben Sie die Steuersignale als Dezimalwerte an.
- Alle **nicht relevanten** Steuerbefehle werden auf 0 gesetzt.
- Sie finden die **lw-Instruktion** in Binär, Dezimal- und Hexadezimalformat auf der nächsten Seite.
- Sie können alle Werte in Binär, Dezimal- oder Hexadezimalformat angeben. Achten Sie aber darauf, dass klar ist, welches Format Sie jeweils verwenden.

6.2 Erklären sie den Unterschied zwischen Latenz und Durchsatz im Bezug auf eine CPU. (1P)

6.3 Was versteht man unter Speed-Up bei einer Pipeline und wie wird dies berechnet? (1P)

6.4 Was bestimmt die Taktzykluszeit bei einer Single Cycle CPU und was bestimmt die Taktzykluszeit bei einer Multi Cycle CPU? (1P)

Name:

Matrikelnummer:

Abbildung 6.1: Code-Ausschnitt

Zeile	Adresse	Instruktion
1	4000	add \$s3, \$s2, \$t0
2	4004	lw \$t3, 4(\$a3)
3	4008	addi \$a3, \$a3, 20
4	4012	add \$s3, \$a3, \$t0
5	4016	lw \$t3, 8(\$s3)

Assembler: lw \$t3, 8(\$s3)
Binär (Format): 001011 10011 01011 00000000000001000
Dezimal (Format): 11 19 11 8
Binär (je 4 Bit): 0010 1110 0110 1011 0000 0000 0000 1000
Hexadezimal: 2e 6b 00 08

Tabelle 6.1: Register- und Speicherinhalt vor Ausführung von Zeile 1

Register	\$a3	\$s1	\$s2	\$t0
Inhalt (Word)	20	8	12	12
Speicher	4	24	40	60
Inhalt (Word)	12	16	12	20

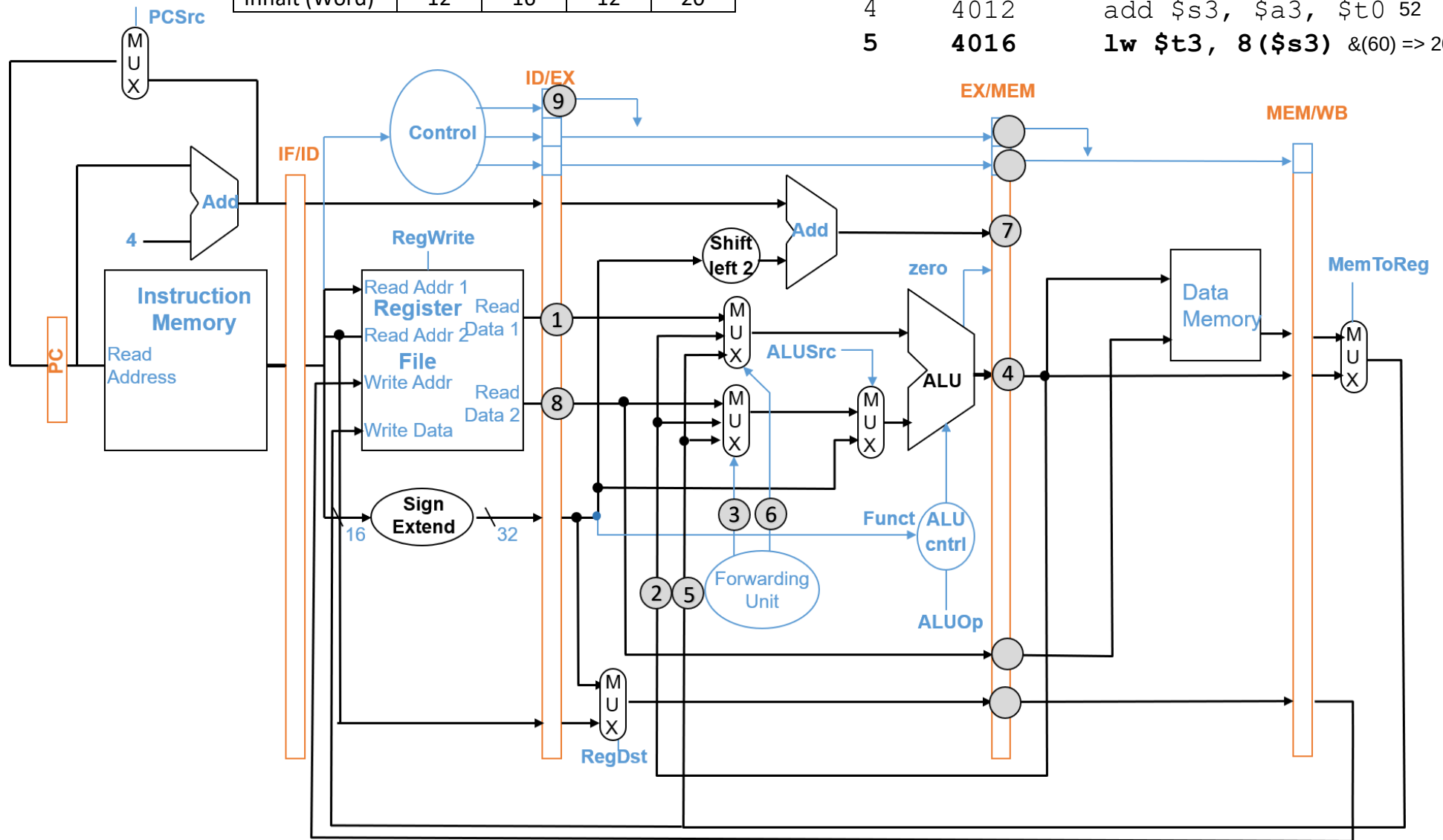
Tabelle 6.2: Registerinhalte, Daten- und Steuersignale bei Ausführung von "lw \$t3, 8(\$s3)" in EX

1		3		5		7	
2		4		6		8	
9							

Name:

Register	\$a3	\$s1	\$s2	\$t0
Inhalt (Word)	20	8	12	12
Speicher	4	24	40	60
Inhalt (Word)	12	16	12	20

Zeile	Adresse	Instruktion
1	4000	add \$s3, \$s2, \$t0 24
2	4004	lw \$t3, 4(\$a3) 16
3	4008	addi \$a3, \$a3, 20 40
4	4012	add \$s3, \$a3, \$t0 52
5	4016	lw \$t3, 8(\$s3) &(60) => 20



1	ID/EX Data1: 52	3	FW Op2: 1	5	FW MEM/WB:	7	
2	FW EX/MEM: 52	4	EX/MEM ALU: 60	6	FW Op1: 0	8	ID/EX Data2: 8
9							

Aufgabe 7: Cache (15 Punkte)

Ein Prozessor verwendet einen getrennten Daten- und Instruktionscache. Der Daten-Cache wird über eine Speicherhierarchie mit drei Cache-Ebenen realisiert. Auf allen drei Cache-Ebenen wird eine gemeinsame Blockgröße von 8 Bytes verwendet. Eine Übersicht der Cache-Hierarchie finden Sie in der folgenden Tabelle:

Level	Größe	Organisation	Schreib-Strategie
First	512 Bytes	4-way-set associative	Write-Back mit Allocate-on-miss
Second	1024 Bytes	2-way-set associative	
Third	65536 Bytes	8-way-set associative	

Ein Auszug des Inhalts der Caches ist in der Tabelle auf der nächsten Seite gegeben. Alle Caches verwenden eine LRU-Ersetzungsstrategie. Die Reihenfolge, in der die Blöcke ersetzt werden, wird über die Position des Blocks in der Tabelle vermerkt. Dabei wird der am weitesten rechts stehende Block zuerst ersetzt.

Die folgende Instruktion wird ausgeführt: `lw $t4, 0($sp)`

Vor der Ausführung steht im `$sp`-Register der folgende Wert:

Dezimal: 74540

Binär: 0000 0000 0000 0001 0010 0011 0010 1100

Hexadezimal: 00 01 23 2c

7.1 Bestimmen Sie, welcher Wert in das Register `$s3` geschrieben wird. Geben Sie alle Zwischenschritte an. (8P)

7.2 Tragen Sie in der leeren Tabelle die **Änderungen** der Cache-Inhalte für den **First und Second Level** Cache ein, die durch die Ausführung der „lw“-Instruktion verursacht werden.

Hinweis: Wenn sich lediglich die Position eines Blocks im Set verändert, genügt es lediglich die Tags an der richtigen Position einzutragen. Ein Kopieren der Inhalte ist nicht notwendig. (7P)

Auszug First-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
1	466	Y	N	5F 60	170	Y	N	43 65	205	Y	N	9D 4E	558	Y	N	EE 5A
2	200	Y	N	B1 95	94	Y	N	75 57	144	Y	N	CF 7D	533	N	N	93 94
3	224	Y	N	55 A3	236	Y	N	10 27	538	Y	N	3C 17	222	Y	N	D3 78
4	523	Y	N	3F 00	539	Y	N	6A E7	977	Y	N	34 A2	365	Y	N	97 81
5	177	Y	N	BD 4B	966	Y	N	28 0E	110	Y	N	40 47	155	Y	Y	26 3A
6	545	Y	N	6A 87	159	Y	N	18 22	225	Y	N	2C DA	611	N	N	C2 35

Auszug Second-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block
33	116	Y	N	5F 60	42	Y	N	43 65
34	23	Y	N	75 57	81	N	N	05 F6
35	134	Y	N	3C 17	55	Y	N	D3 78
36	145	Y	N	CF 2A	254	Y	N	19 39
37	241	Y	N	28 0E	27	Y	Y	40 47

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block
49	241	Y	N	0A 2B	92	Y	N	A5 40
50	23	Y	N	5C 2F	114	Y	N	A1 5D
51	82	Y	N	88 77	21	N	N	0E B8
52	130	Y	N	3F 00	134	Y	N	6A E7
53	156	Y	N	9B A0	34	N	N	44 F7

Auszug Third-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block				
98	5	Y	Y	05 F6	9	Y	N	8B 50	8	Y	Y	3C F1	7	Y	N	86 04	15	Y	Y	BB 71	1	Y	N	52 AF	2	N	N	C3 F4	85	Y	Y	8A 73
99	7	Y	Y	C8 FE	15	Y	Y	E3 A4	5	Y	N	66 8E	1	Y	Y	34 AA	8	Y	Y	BB D9	3	Y	N	38 BF	2	N	N	21 8E	35	Y	Y	99 07
100	9	Y	Y	CF 2A	2	Y	Y	EA 1A	7	Y	Y	52 36	1	Y	Y	B0 AC	8	Y	Y	8A 1E	5	Y	N	16 31	3	Y	Y	2B 14	15	N	Y	69 E0
101	15	Y	Y	28 0E	2	Y	Y	E8 42	7	Y	Y	57 48	3	Y	Y	EC F3	1	Y	Y	B8 E4	5	Y	N	8E EE	8	Y	Y	5A F6	9	Y	N	68 35
102	9	Y	Y	A4 2E	2	Y	N	3E 3A	15	Y	N	64 A9	7	Y	N	D7 78	3	Y	Y	4F 07	5	Y	N	18 80	1	Y	N	DC 63	8	Y	N	C0 C9

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block				
436	8	Y	Y	6A E7	15	Y	N	45 87	3	Y	Y	1A 46	2	Y	Y	1D 03	9	Y	Y	39 B9	1	Y	N	C0 78	5	Y	Y	E2 0F	7	Y	Y	B5 F8
437	9	Y	Y	6E A9	5	Y	Y	D6 5F	1	Y	N	75 03	8	Y	Y	67 DD	15	Y	Y	1E FB	3	Y	N	9B 97	35	Y	Y	F3 3D	98	Y	Y	4C 0E
438	5	Y	Y	C9 35	2	Y	Y	4C 83	15	Y	Y	D4 85	7	Y	N	24 4E	9	N	N	3F E1	74	Y	Y	97 3E	14	Y	Y	47 1A	1	Y	Y	29 43

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
737	8	Y	Y	EE 5A	7	Y	Y	43 EC	15	Y	Y	20 28	2	Y	N	44 D7	9	Y	Y	C8 4B	3	Y	N	56 95	1	N	Y	F1 41	10	Y	N	CC 39
738	5	Y	Y	AB 24	1	Y	N	B6 4F	9	Y	Y	71 FC	3	Y	N	86 F6	8	Y	Y	F6 D1	7	Y	Y	23 8E	2	Y	Y	98 C0	15	Y	N	06 A1
739	15	Y	Y	57 AB	2	Y	Y	CA 41	3	Y	Y	2A 22	5	Y	Y	30 73	9	Y	N	9A 1C	85	Y	N	73 21	5	Y	N	A2 AE	28	Y	Y	14 FC
740	1	Y	Y	5E 84	9	Y	N	24 14	8	Y	Y	43 7F	15	Y	Y	41 58	7	Y	Y	CB 7A	3	Y	Y	C5 1C	5	Y	Y	63 81	2	Y	N	00 EA
741	1	Y	N	40 47	7	Y	Y	83 BF	2	Y	Y	DC EE	8	Y	Y	B8 61	9	Y	Y	7F 91	5	Y	Y	81 34	15	Y	N	73 C2	3	Y	N	2F F5

Auszug First-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
1																
2																
3																
4																
5																
6																

Auszug Second-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block
33								
34								
35								
36								
37								

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block
49								
50								
51								
52								
53								

Auszug Third-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
98																												
99																												
100																												
101																												
102																												

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
436																												
437																												
438																												

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
737																												
738																												
739																												
740																												
741																												